AFRILANG Stdlib

std/c/graphcent

Bibliothèque AFRILANG `std/c/graphcent` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : degreeCentralit

`import "std/c/graphcent"` · `use graphcent`

- `degreeCentrality(adj list number, n number) ? list number`
- `closenessCentrality(adj list number, n number) ? list number`
- `betweennessApprox(adj list number, n number) ? list number`
- `eigenvectorCentrality(adj list number, n number) ? list number`
- `pageRank(adj list number, n number, damping number) ? list number`
- `hubScore(adj list number, n number) ? list number`
- `mostCentral(adj list number, n number) ? number`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphcent` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : degreeCentralit

`import "std/c/graphcent"` · `use graphcent`

- `degreeCentrality(adj list number, n number) ? list number`
- `closenessCentrality(adj list number, n number) ? list number`
- `betweennessApprox(adj list number, n number) ? list number`
- `eigenvectorCentrality(adj list number, n number) ? list number`
- `pageRank(adj list number, n number, damping number) ? list number`
- `hubScore(adj list number, n number) ? list number`
- `mostCentral(adj list number, n number) ? number`

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphcent"
use graphcent
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

? degreeCentrality(adj list number, n number) ? list
? closenessCentrality(adj list number, n number) ? list
? betweennessApprox(adj list number, n number) ? list
? eigenvectorCentrality(adj list number, n number) ? list
? pageRank(adj list number, n number, damping number) ? list
? hubScore(adj list number, n number) ? list
? mostCentral(adj list number, n number) ? number

4. Exemples d'utilisation

```
import "std/c/graphcent"
use graphcent

create result = degreeCentrality(/* args */)
say result

create result = closenessCentrality(/* args */)
say result

create result = betweennessApprox(/* args */)
say result

create result = eigenvectorCentrality(/* args */)
say result

create result = pageRank(/* args */)
say result

create result = hubScore(/* args */)
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/graphcent"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module graphcent
```

```
function degreeCentrality(adj list number, n number) returns list number
end

function closenessCentrality(adj list number, n number) returns list number
end

function betweennessApprox(adj list number, n number) returns list number
end

function eigenvectorCentrality(adj list number, n number) returns list number
end

function pageRank(adj list number, n number, damping number) returns list number
end

function hubScore(adj list number, n number) returns list number
end

function mostCentral(adj list number, n number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphcent` (tier complex, category complex). Exports 7 function(s): degreeCentrality, closenessC

`import "std/c/graphcent"` · `use graphcent`

- `degreeCentrality(adj list number, n number) ? list number`
- `closenessCentrality(adj list number, n number) ? list number`
- `betweennessApprox(adj list number, n number) ? list number`
- `eigenvectorCentrality(adj list number, n number) ? list number`
- `pageRank(adj list number, n number, damping number) ? list number`
- `hubScore(adj list number, n number) ? list number`
- `mostCentral(adj list number, n number) ? number`

2. Installation & import

Add the module to your program:

```
import "std/c/graphcent"
use graphcent
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

? degreeCentrality(adj list number, n number) ? list
? closenessCentrality(adj list number, n number) ? list
? betweennessApprox(adj list number, n number) ? list
? eigenvectorCentrality(adj list number, n number) ? list
? pageRank(adj list number, n number, damping number) ? list
? hubScore(adj list number, n number) ? list
? mostCentral(adj list number, n number) ? number

4. Usage examples

```
import "std/c/graphcent"
use graphcent

create result = degreeCentrality(/* args */)
say result

create result = closenessCentrality(/* args */)
say result

create result = betweennessApprox(/* args */)
say result

create result = eigenvectorCentrality(/* args */)
say result

create result = pageRank(/* args */)
say result

create result = hubScore(/* args */)
say result
```

5. Best practices

? Prefer `import "std/c/graphcent"` at the top of your file.
? Run `afrilang check` before shipping.
? Combine with related stdlib modules from the same category.
? See /stdlib/ for the live source and playground demos.
? Report issues on GitHub if an export is missing or undocumented.

6. Appendix ? source

```
module graphcent
```

```
function degreeCentrality(adj list number, n number) returns list number
end

function closenessCentrality(adj list number, n number) returns list number
end

function betweennessApprox(adj list number, n number) returns list number
end

function eigenvectorCentrality(adj list number, n number) returns list number
end

function pageRank(adj list number, n number, damping number) returns list number
end

function hubScore(adj list number, n number) returns list number
end

function mostCentral(adj list number, n number) returns number
end
end
```